



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO®



Instituto **Tecnológico**®
de Aguascalientes

Instituto Tecnológico de Aguascalientes

Ingeniería en Tecnologías de la Información y las Comunicaciones

Tecnologías Inalámbricas

12:00 p m

Wifi 2 ESP32

Profesor: Ricardo Alejandro Rodríguez Jiménez

Equipo:

Ramírez Rodríguez Martín de Jesús 23151193

Santacruz Rodarte Cristian Alexis 22150108

Pedroza Pérez Emmanuel 23151202

Fecha: 22 de mayo de 2026

Índice

Introducción	3
Código ArduinoIDE	4
Conclusiones	9
Enlace Github	10



Introducción

En esta práctica se desarrolló una solución basada en el ESP32 que combina las funciones de Access Point y portal cautivo con la capacidad de conectarse posteriormente a una red WiFi externa. Inicialmente, el dispositivo genera una red inalámbrica propia que permite a los usuarios acceder a una página de autenticación mediante un portal cautivo. Una vez que las credenciales son validadas correctamente, el ESP32 establece una conexión con una red WiFi previamente configurada, actuando como un dispositivo auxiliar capaz de mantener acceso a Internet mientras continúa proporcionando servicios de red local.



Código ArduinoIDE

El ESP32 ha requerido conexión USB al computador para esta práctica:

Librerías que controlan la función wifi del chip integrado en el esp-32, permite hacer un dns que controle el acceso al portal captivo y levantar un servidor en el esp-32 respectivamente.

```
#include <WiFi.h>
```

```
#include <DNSServer.h>
```

```
#include <WebServer.h>
```

Variables para el nombre de la red a la que el ESP se conectara y la contraseña

```
const char* ssid1 = "estudiantes";
```

```
const char* password1 = "987654312.#";
```

Variables para el nombre de la red AP y la contraseña

```
const char* ssid = "ichi ni san";
```

```
const char* password = "";
```

Se asigna que IP usara el Access Point

```
IPAddress apIP(192,168,4,1);
```

Levanta el DNS y el servidor web en el puerto 80

```
DNSServer dnsServer;
```

```
WebServer webServer(80);
```

Variables para el usuario y contraseña del portal captivo

```
const char* validUser = "admin";
```

```
const char* validPass = "1234";
```

Función para el portal captivo donde esta construido la pagina html con el formulario de acceso

```
void PortalCaptivo() {  
    String html = "<!DOCTYPE html><html><head><title>Portal Captivo</title></head><body>";  
    html += "<h1>Identificarse</h1>";  
    html += "<form action='/login' method='POST'>";  
    html += "Usuario: <input type='text' name='usuario'><br>";  
    html += "Contraseña: <input type='password' name='contraseña'><br>";  
    html += "<input type='submit' value='Login'>";  
    html += "</form></body></html>";  
    webServer.send(200, "text/html", html);  
}
```

Funcion Login que verifica los datos de acceso del portal captivo. Si son correctos pasa a darle la bienvenida al administrador, si son incorrectos manda mensaje de error de identificacion

```
void Login() {  
    if (webServer.hasArg("usuario") && webServer.hasArg("contraseña")) {  
        String usuario = webServer.arg("usuario");  
        String contra = webServer.arg("contraseña");  
  
        if (usuario == validUser && contra == validPass) {  
            webServer.send(200, "text/html", "<h1>Bienvenido administrador</h1><p>ESP32-Access  
Point.</p>");  
            valid();  
        } else {
```

```
webServer.send(200, "text/html", "<h1>Credenciales invalidas</h1><p>Ingresa los datos  
correctamente.</p><a href='/'>Back</a>");
```

```
}
```

```
} else {
```

```
webServer.send(400, "text/html", "error");
```

```
}
```

```
}
```

```
void valid() {
```

Detiene el servidor dns

```
dnsServer.stop();
```

```
Serial.begin(115200);
```

Llama a la function wifi.begin conectandose a la red definida en ssid1 con la contraseña password1

```
WiFi.begin(ssid1, password1);
```

Mensaje que indica que esta intentando conectarse

```
Serial.println("Establishing connection to WiFi with SSID: " + String(ssid1));
```

```
while (WiFi.status() != WL_CONNECTED) {
```

```
delay(1000);
```

```
Serial.print(".");
```

```
}
```

Una vez validada la contraseña da mensaje de conexión exitosa

```
Serial.print("Connected to network with IP address: ");
```

```
Serial.println(WiFi.localIP());
```

```
}
```

```
void setup() {
```

```
  Serial.begin(115200);
```

Inicia el ESP-32 como un router para wifi, usa la IP asignada y usa una mascara de 24 bits

```
  WiFi.softAP(ssid, password);
```

```
  WiFi.softAPConfig(apIP, apIP, IPAddress(255,255,255,0));
```

```
  Serial.println("AP started");
```

```
  Serial.println(WiFi.softAPIP());
```

Inicia el servidor DNS, redirige todos los dominios a la pagina html del servidor del ESP-32

```
  dnsServer.start(53, "*", apIP);
```

Cuando se entra a la dirección del ESP-32 lo dirige al portal captivo. Posteriormente cuando se envia el formulario se dirige al método Login()

```
  webServer.on("/", PortalCaptivo);
```

```
  webServer.on("/login", HTTP_POST, Login);
```

Se redirige el usuario al portal captivo si intenta abrir otra pagina web

```
  webServer.onNotFound(PortalCaptivo);
```

Inicia el servidor web

```
  webServer.begin();
```

```
}
```



```
void loop() {
```

Constantemente está funcionando el DNS para redirigir el tráfico web y atiende los navegadores web que se conectan

```
    dnsServer.processNextRequest();
```

```
    webServer.handleClient();
```

```
}
```



Conclusiones

Martín de Jesús Ramírez Rodríguez:

Se logró implementar un sistema en el ESP32 capaz de crear una red WiFi con portal cautivo para la autenticación de usuarios y, tras validar las credenciales, conectarse automáticamente a una red WiFi. El resultado demuestra la versatilidad del ESP32 para desarrollar soluciones de conectividad inalámbrica con control de acceso y enlace a Internet.

Aunque no había acceso total al internet, fue posible usar ciertos puertos, como para conectarse a WhatsApp.

Pedroza Pérez Emmanuel:

En esta actividad se logra conectar un ESP32 a una red wifi permitiendo que sea usuario o intermediario de alguna aplicación para el paso de datos. Nos permite tener más opciones en la transmisión de datos

Cristian Alexis Santacruz Rodarte:

Con el ESP32 Wifi se hizo la práctica de manera correcta, se implementó una red por medio del chip para la creación de una red wifi con la cual se podía acceder por medio de esta, con la práctica se pudo reforzar los conocimientos en redes por medio de Wif.

Enlace Github

A continuación, se adjunta el enlace con el código ArduinoIDE expuesto, así como el video de evidencia y documentación:

<https://github.com/MartinTAAgs/WIFI-ESP32>

